

Gram-GAN Methodology vs Implementation

Model architecture: The Gram-GAN paper specifies that the *generator* should be an ESRGAN-style RRDB network, and the *discriminator* a VGG-style convolutional network ¹. In particular, the authors state “the generator uses the RRDB [11]... and the discriminator uses a variant of the VGG network” ¹. Any faithful implementation must replicate this. If the provided code instead uses a different generator (e.g. SRResNet or a smaller model) or a standard ESRGAN discriminator, it diverges from the paper. Likewise, the discriminator should output feature maps at intermediate layers (not just the final score) so that those features can be used in the perceptual loss ². In summary, the network should match the ESRGAN backbone for the generator and a VGG-style ConvNet for the discriminator as described ¹.

Patch-wise texture (Gram) loss: Gram-GAN's key novelty is one-to-many supervision by matching textures via Gram matrices. Concretely, for each predicted patch e_i the method constructs a *Gram matrix* $G(p) = p^{\top} p$ from original feature vectors of patches ³. The paper forms a **candidate database** of patches containing (1) the ground-truth patch, (2) a downsampled ground-truth patch, and (3) randomly *affine-warped* versions of the ground-truth patch ⁴ ⁵. It then selects the candidate patch whose Gram matrix is closest (in Euclidean norm) to the Gram of the predicted patch ⁶ ⁴. The “patch-wise texture loss” is then computed between the predicted patch and this best candidate patch ⁵.

If the implementation omits this mechanism, it fails to capture the one-to-many supervision. For example, if the code simply uses the ground-truth patch (one-to-one) or selects a patch by raw pixel distance rather than Gram distance, it contradicts the paper's method ⁷ ⁵. A faithful implementation should **compute Gram matrices for each patch**, include affine transformations of the GT patch in the candidate set, and then enforce a loss $L_{\text{patch}} = \|G(e_i) - G(p_i^*)\|^2$, where p_i^* is the selected patch as described (see Eq.4 in the paper) ⁵ ³. If any of these steps (Gram computation, patch database construction, affine augmentation) are missing or simplified, the implementation deviates from the published method.

Discriminator perceptual loss: Gram-GAN adds a novel perceptual loss using the discriminator's hidden features. The paper states: “the discriminator in each iteration is used to extract features, and the discriminator perceptual loss... is represented as (5)” ². In practice, this means selecting certain convolutional layers of D (the paper uses the 5th and 11th conv layers) and penalizing differences between real and fake features at those layers. An implementation should forward both the generator output and the ground truth through D, capture the specified intermediate feature maps, and include a loss term $L_{\text{disc_perc}} = \sum_k \|D_k(G(z)) - D_k(\text{GT})\|^2$. If the code does not extract and compare discriminator features (for example, if it only uses the final discriminator score), then it omits this loss component. The absence of this term would contradict the claim that “discriminator features can highlight differences... to improve quality” ⁸. A correct implementation must therefore hook the discriminator's 5th and 11th layer activations (after their nonlinearities) and include their MSE difference in the generator's loss, as per the paper ².

VGG perceptual and content losses: In addition to discriminator features, the model uses traditional VGG-based perceptual loss and an L_1 content loss. The VGG loss compares features from pretrained VGG19

(layers conv2_2, conv4_2, conv4_4 with weights 1.0, 0.1, 0.01) ⁹. The content loss is simply the ℓ_1 distance between SR and HR images ¹⁰. Any faithful code should implement these exactly: e.g. using a VGG19 model (frozen) and weighting each feature layer as specified. If the implementation uses different layers or omits the VGG loss entirely, that is a simplification. Likewise, replacing ℓ_1 with ℓ_2 would conflict with the rationale (the paper explicitly justifies ℓ_1 to be “insensitive to outliers” in GAN training) ¹⁰. These losses must be included with the paper’s stated weights in the overall objective (Eq.12) ¹¹.

Adversarial (RaGAN) loss: Gram-GAN employs a **relativistic average GAN** loss with a special masking scheme ¹² ¹³. Specifically, Eqs.(7–9) in the paper define a Relativistic Average GAN discriminator loss, and Eq.(10) defines a binary mask $M(x)=\mathbf{1}\{\sigma(\Omega(x))>\epsilon\}$ based on the standard deviation σ of a random patch of the image. In each iteration, real and fake images are partially masked (the “true” patch is masked out) according to M before computing the GAN loss ¹³. The implementation should therefore (1) compute this random patch mask at each step, (2) apply it to real and SR images, and (3) use the RaGAN loss formulas. If the provided code instead uses a plain GAN loss (e.g. non-relativistic BCE) or neglects the mask, it departs from the method. For example, the mask generation (std threshold = 0.005, patch size 4×4) is quite unusual and unlikely to appear unless explicitly coded. Any code lacking this mechanism should be noted as missing a key component of the SR-GAN design ¹³.

Training strategy and scheduler: The paper reports training with Adam ($\beta_1=0.9, \beta_2=0.999$), batch size 8, input patches of size 48×48 (HR 192×192), with random flips/rotations ¹⁴. Critically, they use a **cyclical learning rate**: 600k iterations total, restarting every 200k, with warm-up and cosine decay from an initial 1×10^{-4} ¹⁵. Implementation should mirror this: e.g. a PyTorch scheduler like `CosineAnnealingWarmRestarts` or a manual warm-up schedule. If the code uses a fixed learning rate or a simple step decay, that is a simplification. It should also perform the stated data augmentation (flip/rotate), which is standard. The mention that “the magnitude of random affine transform was 0.003” ¹⁵ likely refers to the slight affine warp of patches in the supervision database – if the code omits generating random affine variations for patch supervision, this is another missing detail.

Patch data handling: According to the paper, DIV2K training images were cropped by a sliding window into non-overlapping 192×192 patches (yielding 44,226 patches) ¹⁶. The code should either pre-crop the dataset or generate equivalent random crops on-the-fly. More importantly, for the one-to-many patch supervision, the paper implies constructing a small per-patch database (GT patch, down-sampled, affined). If the implementation simply loads HR patches and never explicitly forms this “patch database,” then the one-to-many mechanism isn’t realized. In summary, the code must handle patches of size 4×4 (for mask and Gram) and include the generation of affine variants of each GT patch for supervision ¹⁵ ⁴; absence of these steps is inconsistent with the method.

Inconsistencies and missing components: Based on the above points, any deviations in the code are critical. In particular, if the code lacks the Gram-matrix-based patch selection, it has effectively reduced the model to one-to-one supervision (like a standard SRGAN) rather than the intended one-to-many ⁷ ⁵. If discriminator perceptual loss is not implemented, the “discriminator perceptual loss” contribution is missing ². If the adversarial mask is omitted, the RaGAN loss used is simpler than claimed ¹³. And if the LR schedule does not use warm-up and cosine restarts, training dynamics differ from the paper’s description ¹⁷.

Recommendations: To fully match the paper, the implementation should be modified as follows:

- **Patch supervision:** Compute $G(e_i)$ and $G(p)$ (Gram matrices) and choose the patch p^* minimizing $|G(e_i) - G(p)|$ from the candidate set (GT, downsampled GT, and randomly affined GT patches) ³ ⁵. Add this patch-wise loss $L_{\text{patch}} = |e_i - p^*|^2$ (or Gram-difference) to the generator loss as in Eq.(4). Generate affine variants by applying small random affine warps (variance 0.003) to the GT patch ⁴.
- **Discriminator features:** Hook the discriminator's 5th and 11th convolutional layer outputs during each forward pass. Compute $L_{\text{disc_perc}} = |D_5(G(z)) - D_5(\text{GT})|^2 + |D_{11}(G(z)) - D_{11}(\text{GT})|^2$ and add it (with appropriate weight) to the total generator loss ². Ensure gradients do not update D during this loss.
- **Adversarial mask:** For each real/fake image pair, select a random 4×4 patch location, compute its pixel-std, and form the binary mask $M(x) = \mathbf{1}_{\{\sigma(\text{patch}) > 0.005\}}$. Apply M to the real image and $1-M$ to the fake (or vice versa, as per Eq.9). Then compute the RaGAN losses using masked inputs ¹³. In PyTorch, this could be done by elementwise multiplying images by the mask. This implements Eqs.(7–10).
- **Learning rate schedule:** Use an Adam optimizer ($\beta_1=0.9, \beta_2=0.999$) with an LR scheduler that performs warm-up from 0 to 10^{-4} , then cosine decay resetting every 200k iterations ¹⁵.

By adding these components, the code will faithfully implement the Gram-GAN methodology. Failure to include any of the above would mean the implementation only partially realizes the paper's contributions. Each modification above is grounded in the original description ⁵ ² ¹³ and is necessary to replicate the reported results.

Sources: The analysis above references the Gram-GAN paper's descriptions of architecture, losses, and training protocol ¹ ³ ⁵ ² ⁹ ¹² ¹⁰ ¹⁵, which should be compared line-by-line against the implementation to identify any inconsistencies.

¹ ² ³ ⁴ ⁵ ⁶ ⁷ ⁸ ⁹ ¹⁰ ¹¹ ¹² ¹³ ¹⁴ ¹⁵ ¹⁶ ¹⁷ Gram-GAN: Image Super-Resolution Based on Gram Matrix and Discriminator Perceptual Loss
<https://www.mdpi.com/1424-8220/23/4/2098>